

Analisis del binario

reto.exe

y desarrollo de un keygen

AUTOR	Francisco Serrano Sanchez	
PRACTICA	Ejercicio 1 — Ingenieria Inversa de binarios	
ENTREGABLE	Informe en PDF + Keygen (Python y C)	
BINARIO ANALIZADO	reto.exe	PE32 ejecutable de consola (Intel x86, 32 bits)
HASH SHA-256	6b9d4641e42cef95bc2547154566b9cb350cab3fecf6cf7f05087d055e094039	

1 Introduccion y objetivos

El presente informe documenta el proceso de ingenieria inversa aplicado al binario `reto.exe`, un ejecutable de consola para Windows que implementa un sencillo «Sistema de Registro». El programa solicita al usuario un nombre y una clave de registro y, segun la validez de esta ultima, muestra el mensaje «Registro exitoso!» o «Registro fallido!».

El enunciado de la practica (`ejercicio1.txt`) plantea dos objetivos:

1. **Analizar el binario** y describir las operaciones que realiza, en particular el mecanismo de comprobacion de la clave de licencia.
2. **Emular la generacion de claves** mediante un programa (*keygen*) capaz de producir una licencia valida para cualquier usuario, evitando asi el coste de la licencia.

Ambos objetivos se han cumplido: el algoritmo de validacion ha sido reconstruido por completo y se ha desarrollado un keygen funcional, entregado en dos lenguajes (Python y C). Este documento detalla los pasos seguidos, las observaciones realizadas durante el analisis y la solucion final adoptada.

NOTA SOBRE EL AMBITO DE LA PRACTICA

`reto.exe` es un binario de tipo *crackme*, es decir, un ejercicio creado deliberadamente para ser analizado con fines didacticos. No se trata de software comercial real, sino de un programa de practicas cuya finalidad es ejercitar las tecnicas de analisis estatico de codigo.

2 Metodologia y herramientas

El analisis se ha realizado siguiendo un enfoque de **analisis estatico**, es decir, sin ejecutar el binario, examinando su estructura, sus cadenas de texto y su codigo desensamblado. Las fases del trabajo han sido las siguientes:

#	Fase	Descripcion
1	Identificacion	Determinacion del formato del fichero, arquitectura y compilador.
2	Inspeccion de cadenas	Extraccion de las cadenas de texto para localizar mensajes y pistas.
3	Analisis de importaciones	Estudio de las funciones de libreria usadas (IAT) para inferir la logica.
4	Desensamblado	Traduccion del codigo maquina de la seccion <code>.text</code> a ensamblador x86.
5	Reconstruccion	Interpretacion de cada rutina y reconstruccion del algoritmo en pseudocodigo.
6	Implementacion	Desarrollo del keygen y verificacion frente a la logica del binario.

Las herramientas empleadas han sido utilidades estandar de la linea de comandos:

- `file` — identificacion del formato del ejecutable.
- `strings` — extraccion de cadenas ASCII y Unicode.
- `objdump` — volcado de cabeceras PE, secciones, tabla de importaciones y desensamblado en sintaxis Intel.
- `Python 3` — emulacion del algoritmo e implementacion del keygen.

3 Informacion general del binario

El fichero `reto.exe` es un ejecutable **PE32** (Portable Executable de 32 bits) de consola para Microsoft Windows, compilado para la arquitectura Intel **x86**. No presenta indicios de empaquetado ni de ofuscacion: el codigo es directamente legible y las cadenas aparecen en claro.

Propiedad	Valor
Nombre	reto.exe
Formato	PE32 ejecutable (consola), Intel 80386
Tamano	11.264 bytes
Numero de secciones	5 (.text, .rdata, .data, .rsrc, .reloc)
MD5	f6c673d8bfd2bb6d930fa86864862ea0
SHA-1	4866d5d6087029630b718320b26e96c1e162b724
SHA-256	6b9d4641e42cef95bc2547154566b9cb350cab3fecf6cf7f05087d055e094039
Compilador	Microsoft Visual C++ (MSVC), runtime VCRUNTIME140.dll
Ruta del PDB	C:\Users\User\source\repos\ConsoleApplication1\Release\ConsoleApplication1.pdb

La ruta del fichero de simbolos (`.pdb`) embebida en el binario revela que el proyecto original se llamaba *ConsoleApplication1* y fue compilado en configuracion **Release**, lo que confirma que se trata de una aplicacion de consola escrita en C/C++.

3.1. Mapa de secciones

Seccion	VA inicio	Tamano	Contenido
.text	0x00401000	0x1014	Codigo ejecutable
.rdata	0x00403000	0x0F98	Datos de solo lectura: cadenas e IAT
.data	0x00404000	0x0200	Datos: contiene la tabla de la clave
.rsrc	0x00405000	0x01E0	Recursos (manifiesto de la aplicacion)
.reloc	0x00406000	0x017C	Tabla de reubicaciones

OBSERVACION: AUSENCIA DE PROTECCIONES

El binario importa `IsDebuggerPresent` desde `KERNEL32.dll` , pero **no se trata de una proteccion anti-depuracion activa**: esa importacion forma parte del codigo de arranque estandar del runtime de MSVC (gestion de excepciones de seguridad). La logica de registro propiamente dicha no contiene ninguna comprobacion anti-debug, anti-VM ni cifrado.

4 Analisis estatico: cadenas e importaciones

4.1. Cadenas de texto relevantes

La extraccion de cadenas revela los mensajes que el programa muestra por pantalla, asi como el formato usado para leer la entrada del usuario:

```
# Cadenas localizadas en la seccion .rdata
"Bienvenido al sistema de Registro."
"Por favor, introduzca su nombre de usuario "
"%10s" ; formato de lectura (scanf)
"Ahora, introduzca su clave de registro: "
"%10s"
"Registro exitoso!.."
"Registro fallido! Intenta nuevamente."
"R3V3RS1NG!" ; <-- cadena sospechosa (ver 6.4)
```

Dos detalles destacan ya en esta fase. En primer lugar, la cadena de formato `"%10s"` indica que tanto el nombre de usuario como la clave se leen con `scanf`, limitados a un maximo de 10 caracteres y **sin espacios** (el especificador `%s` se detiene en el primer caracter en blanco). En segundo lugar, aparece la cadena `"R3V3RS1NG!"`, que no corresponde a ningun mensaje mostrado al usuario y que, como se vera, es la clave del algoritmo.

4.2. Funciones importadas (IAT)

La tabla de importaciones muestra que funciones de libreria utiliza el binario. Las relevantes para la logica de registro son:

Funcion	Libreria	Papel en el programa
<code>__stdio_common_vfscanf</code>	api-ms-win-crt-stdio	Base de <code>scanf</code> : lectura del usuario y la clave.
<code>__stdio_common_vfprintf</code>	api-ms-win-crt-stdio	Base de <code>printf</code> : impresion de mensajes.
<code>isdigit</code>	api-ms-win-crt-string	Comprueba caracter a caracter que la clave sea numerica.
<code>atoi</code>	api-ms-win-crt-convert	Convierte la clave (texto) a un numero entero.
<code>getchar</code>	api-ms-win-crt-stdio	Pausa final para que la ventana no se cierre.

La combinacion de `isdigit` + `atoi` es muy reveladora: anticipa que la clave de registro debe ser una **cadena puramente numerica** que el programa transforma en un entero y compara con un valor calculado internamente.

5 Analisis del flujo de ejecucion

La funcion principal se encuentra en la direccion `0x401180`. Su estructura, reconstruida a partir del desensamblado, es la siguiente:

```
; ---- main() @ 0x401180 (reconstruccion) ----
printf("Bienvenido al sistema de Registro.");
printf("Por favor, introduzca su nombre de usuario ");
scanf("%10s", usuario); ; buffer en [ebp-0x1C]

resultado = sub_401000(usuario); ; calcula el valor 'S' del usuario
if (resultado == -1) { ; usuario no valido
    printf("Registro fallido! Intenta nuevamente.");
    return 0;
}

printf("Ahora, introduzca su clave de registro: ");
scanf("%10s", clave); ; buffer en [ebp-0x10]

if (sub_401130(clave) == 0) { ; la clave NO es numerica
    printf("Registro fallido! Intenta nuevamente.");
    return 0;
}

valor_esperado = resultado ^ 0x6667; ; <-- XOR con constante
if (sub_4010A0(atoi(clave), valor_esperado) != 0)
    printf("Registro exitoso!.."); ; CLAVE CORRECTA
else
    printf("Registro fallido! Intenta nuevamente.");
```

El flujo identifica con claridad **tres comprobaciones encadenadas** y tres subrutinas auxiliares que deben analizarse en detalle:

- `sub_401000` — valida el nombre de usuario y calcula un valor numerico a partir de el.
- `sub_401130` — verifica que la clave introducida este compuesta solo por digitos.
- `sub_4010A0` — compara la clave introducida con el valor esperado.

Ademas, se observa la operacion clave del algoritmo: el valor calculado del usuario se combina mediante un **XOR con la constante** `0x6667` antes de la comparacion final.

6 Analisis detallado de las subrutinas

6.1. Subrutina sub_401000 — validacion y calculo del usuario

Esta funcion recibe el nombre de usuario y devuelve un valor entero `S`. Primero recorre la cadena para calcular su longitud y, si esta **no es exactamente 10**, devuelve `-1` (registro fallido). En caso contrario, recorre los 10 caracteres acumulando una suma:

```
401038 cmp [ebp-0x14], 0x0A ; longitud == 10 ?
401042 je 0x401049
401044 or eax, 0xFFFFFFFF ; longitud != 10 -> return -1
...
401069 movsx ecx, byte [usuario+i] ; ecx = ASCII del caracter i
40106F ... div 0x0A ; edx = i % 10
401078 movzx edx, byte [edx+0x404088] ; edx = TABLA[i % 10]
40107F add edx, [ebp-0x10] ; acumulador
401082 add edx, ecx ; acumulador += TABLA[i] + ASCII(char)
401084 mov [ebp-0x10], edx
```

En cada iteracion `i` se suma al acumulador el valor ASCII del caracter `usuario[i]` mas el byte `TABLA[i % 10]`, donde `TABLA` es un vector situado en la direccion `0x404088`. Como el usuario tiene siempre 10 caracteres, el indice `i % 10` coincide con `i`, por lo que se recorre la tabla entera una sola vez. El valor devuelto es:

$$S = \sum \text{ASCII}(\text{usuario}[i]) + \sum \text{TABLA}[i] \quad (i = 0..9)$$

6.2. Subrutina sub_4010A0 — comparacion de claves

Recibe dos enteros y los compara **digito a digito**: en un bucle extrae el ultimo digito de cada numero (resto de la division entre 10) y, si en algun momento difieren, devuelve `0`. Al terminar, si ambos numeros se han consumido por completo (ambos llegan a cero), devuelve `1`. Funcionalmente equivale a una **comparacion de igualdad** entre los dos enteros:

```
; sub_4010A0(a, b) equivale a:
return (a == b) ? 1 : 0;
```

6.3. Subrutina sub_401130 — verificacion numerica de la clave

Recorre la cadena de la clave caracter a caracter y llama a `isdigit` sobre cada uno. Si encuentra un caracter que no es un digito, devuelve `0`; si llega al final sin incidencias, devuelve `1`. Su unico cometido es garantizar que la clave introducida sea un **numero entero en formato texto**.

6.4. La tabla oculta y la constante XOR

El elemento decisivo del algoritmo es la `TABLA` situada en `0x404088`. El volcado de la seccion `.data` en esa direccion muestra exactamente 10 bytes:

```
404088 52 33 56 33 52 53 31 4E 47 21 ; -> "R3V3RS1NG!"
```

Es decir, la tabla no es mas que los codigos ASCII de la cadena «**R3V3RS1NG!**» hallada en el analisis de cadenas. La suma de sus 10 bytes es una **constante**:

Caracter	R	3	V	3	R	S	1	N	G	!	Suma
ASCII	82	51	86	51	82	83	49	78	71	33	666

Por tanto, $\sum \text{TABLA}[i] = 666$. Por otra parte, la instruccion de `main` en `0x401228` aplica un XOR con la constante `0x6667` (26215 en decimal) sobre el valor `S` antes de comparar:

```
401225 mov ecx, [ebp-0x04] ; ecx = S
401228 xor ecx, 0x6667 ; ecx = S XOR 26215 = valor_esperado
```

7 Reconstruccion del algoritmo de licencia

Reuniendo todas las piezas, el procedimiento completo de validacion que realiza `reto.exe` es el siguiente:

- Se lee el **nombre de usuario**. Debe tener **exactamente 10 caracteres** y no contener espacios; en caso contrario, el registro falla de inmediato.
- Se calcula el valor interno `S = Σ ASCII(usuario) + 666`, donde 666 es la suma fija de la tabla «R3V3RS1NG!».
- Se lee la **clave de registro**. Debe ser una cadena compuesta unicamente por **digitos**.
- La clave se convierte a entero con `atoi` y el registro es **exitoso si, y solo si**, ese entero es igual a `S XOR 0x6667`.

ALGORITMO DE GENERACION DE LA CLAVE VALIDA

Despejando, la clave correcta para cualquier nombre de usuario de 10 caracteres es un numero entero que se obtiene directamente con la siguiente formula:

$$CLAVE = (\sum ASCII(usuario) + 666) \text{ XOR } 26215$$

Se trata, por tanto, de un esquema de licencia **completamente determinista y reversible**: a cada nombre de usuario le corresponde una unica clave, y dicha clave puede calcularse sin necesidad de fuerza bruta. El uso de la cadena «R3V3RS1NG!» y de las constantes 666 / 0x6667 no aporta seguridad criptografica real; son simplemente un ofuscamiento ligero pensado para el ejercicio.

7.1. Ejemplo de calculo paso a paso

Para el usuario `FranSerran` (10 caracteres):

- Suma ASCII del usuario: F+r+a+n+S+e+r+r+a+n = **1010**
- Suma de la tabla: **666**
- Valor interno: $S = 1010 + 666 = 1676$
- Clave valida: $1676 \text{ XOR } 26215 = 24811$

Introduciendo el usuario `FranSerran` y la clave `24811`, el binario muestra «Registro exitoso!».

8 Diseno e implementacion del keygen

A partir del algoritmo reconstruido se ha desarrollado un **keygen** que emula la generacion de claves. Se entrega en dos lenguajes — Python y C — para mayor portabilidad; el codigo C reproduce ademas el lenguaje del binario original. La logica esencial, identica en ambos, es trivial:

```
# Nucleo del keygen (Python)
LOOKUP_TABLE = "R3V3RS1NG!"      # tabla en VA 0x404088
XOR_CONST     = 0x6667            # constante XOR (26215)

def generar_clave(usuario):
    if len(usuario) != 10:
        raise ValueError("El usuario debe tener 10 caracteres")
    s = sum(ord(c) for c in usuario) + sum(ord(c) for c in LOOKUP_TABLE)
    return s ^ XOR_CONST
```

El programa entregado incluye ademas validacion de la entrada (longitud y ausencia de espacios), un modo interactivo y un modo por argumento de linea de comandos, y una funcion de **autocomprobacion** que reproduce internamente la verificacion del binario para confirmar que la clave generada seria aceptada. El codigo fuente completo se incluye en el **Anexo A**.

USO DEL KEYGEN

Python: `python3 keygen.py NOMBREUSER`

C: `gcc -O2 -o keygen keygen.c` y luego `./keygen NOMBREUSER`

Donde `NOMBREUSER` es un nombre de exactamente 10 caracteres sin espacios. Si no se indica argumento, el programa lo solicita de forma interactiva.

9 Pruebas y verificacion

Para validar el keygen se han generado claves para distintos nombres de usuario y se ha comprobado el resultado emulando la logica exacta del binario. Todos los casos producen «Registro exitoso!»:

Usuario (10 car.)	Suma ASCII	S = suma + 666	Clave (S XOR 26215)	Resultado
FranSerran	1010	1676	24811	Registro exitoso
FSerrano01	897	1563	24700	Registro exitoso
ABCDEFGHJI	695	1361	25398	Registro exitoso
0123456789	525	1191	25280	Registro exitoso
reversing1	1030	1696	24775	Registro exitoso

Tambien se han verificado los **casos negativos**, comprobando que el binario rechaza correctamente las entradas invalidas, en coherencia con el analisis:

- Usuario con longitud distinta de 10 → *Registro fallido* (lo rechaza `sub_401000`).
- Clave con caracteres no numericos → *Registro fallido* (lo rechaza `sub_401130`).
- Clave numerica pero incorrecta → *Registro fallido* (lo rechaza `sub_4010A0`).

A Anexo A — Código fuente del keygen (Python)

Fichero `keygen.py` . Se entrega tambien la version en C (`keygen.c`) junto a este informe.

```
#!/usr/bin/env python3
# keygen.py -- Generador de claves para reto.exe
# Autor: Francisco Serrano Sanchez

LOOKUP_TABLE = "R3V3RS1NG!"      # 10 bytes en VA 0x404088 (.data)
XOR_CONST     = 0x6667           # constante XOR (= 26215)
USER_LEN      = 10               # longitud exacta exigida
TABLE_SUM     = sum(ord(c) for c in LOOKUP_TABLE)  # == 666

def generar_clave(usuario):
    """Devuelve la clave numerica valida para el usuario indicado."""
    if len(usuario) != USER_LEN:
        raise ValueError("El usuario debe tener EXACTAMENTE 10 caracteres")
    if any(c.isspace() for c in usuario):
        raise ValueError("El usuario no puede contener espacios")
    s = sum(ord(c) for c in usuario) + TABLE_SUM
    return s ^ XOR_CONST

def verificar(usuario, clave):
    """Emula la comprobacion del binario (autocomprobacion)."""
    if len(usuario) != USER_LEN or not clave.isdigit():
        return False
    s = sum(ord(c) for c in usuario) + TABLE_SUM
    return int(clave) == (s ^ XOR_CONST)

if __name__ == "__main__":
    import sys
    usuario = sys.argv[1] if len(sys.argv) > 1 \
        else input("Usuario (10 caracteres): ").strip()

    try:
        clave = generar_clave(usuario)
    except ValueError as err:
        print("[ERROR]", err); sys.exit(1)
    print("Usuario :", usuario)
    print("Clave   :", clave)
    print("Valida  :", verificar(usuario, str(clave)))
```

A.1. Tabla resumen de direcciones relevantes

Direccion	Elemento	Descripcion
0x401000	sub_401000	Valida el usuario (longitud 10) y calcula S.
0x4010A0	sub_4010A0	Compara dos enteros (igualdad).
0x401130	sub_401130	Comprueba que la clave sea numerica.
0x401180	main	Funcion principal: orquesta el registro.
0x401228	instruccion XOR	<code>xor ecx, 0x6667</code> sobre el valor S.
0x404088	TABLA	10 bytes = "R3V3RS1NG!" (suma = 666).